APPENDIX A
EXPERIMENTAL SETTINGS

In our experimental setup, we performed tests on several datasets to evaluate performance. The first dataset, ogbg-ppa [36], consists of undirected protein association neighborhoods derived from the protein-protein association networks of 1,581 species, spanning 37 broad taxonomic groups. Another dataset, ENZYMES [3], includes 600 protein tertiary structures from the BRENDA enzyme database, representing six different enzyme classes. The MSRC_21 dataset [42] contains 563 graphs across 20 categories, with an average of 77.52 nodes per graph. The PROTEINS [3] dataset is a binary classification set with 1,113 graphs, having an average node count of 39.06 per graph. The MUTAG [43] dataset is another small binary graph dataset, consisting of 188 graphs, each with an average of 18 nodes. The IMDB-BINARY [44] dataset, as the name suggests, is a binary graph classification dataset containing 1,000 graphs with an average node count of 20 per graph, and no node features. Similarly, the REDDIT-MULTI-12K dataset [44] includes 11,929 graphs spread across 11 classes, with an average of 391 nodes per graph and no node features.

Additionally, we utilized the MNIST graph dataset, which is derived from the MNIST computer vision dataset. This dataset contains 55,000 images divided into 10 classes, where each image is represented as a graph.

Our experimental investigations were primarily conducted employing Graph Convolutional Networks (GCN) [40] and Graph Isomorphism Networks (GIN) [37] networks. Notably, the experimental methodology adopted possesses a generality that extends to encompass all Message Passing Neural Networks (MPNNs). Our study centers on observing the learning dynamics of networks during graph classification, particularly examining their adaptability to label noise. We aim to enhance robustness by employing tailored loss functions. Notably, the selection of hyperparameters remains unrestricted, as these parameters depend on both the model architecture and the dataset employed, ensuring a nuanced and generalized approach. In each experiment, we initialize with hyperparameters suited for clean, non-noisy conditions, ensuring optimal model performance. These parameters are subsequently held constant as we introduce varying levels of noise, sample density, or graph order. This approach ensures fair comparisons across experiments and facilitates a comprehensive exploration of model capacities. The synthetic label noise is generated following the methodologies described in [45] and [46] which are considered to be standard techniques for generating synthetic label noise.

A. Hyperparameters

We employed the standard GIN [37] and GCN [40] architectures for most of our experiments. However, to investigate the impact of positive eigenvalues on weight matrices, as outlined in VI, we applied targeted modifications to both the GIN and GCN models C-B.

The table below summarizes the key hyperparameters used for the experiments.

Parameter	Value
Architecture	GCN, GIN
Learning Rate	0.001
Optimizer	Adam,
Batch Size	32
Loss Function	CrossEntropy, SOP and GCOD
Epochs	200(PPA) to 1000
Noise Percentage	0% to 40%
Weight Decay	1e-4
Evaluation Metric	Accuracy
GNN Layers	5
Learning Rate u	1
Hidden Units	300

TABLE VII
NETWORK ARCHITECTURE AND HYPERPARAMETERS.

All experiments have been performed over NVIDIA RTX A6000 GPU. For implementation, visit the following anonymous GitHub: Robustness Graph Classification Project.

APPENDIX B
ADDITIONAL RELATED WORKS

Learning under label noise. Some methods focus on sample relabelling instead [47], [48]. Another family of techniques address noisy labels using two networks, splitting the training set and training two models for mutual assessment [49]–[51]. Regarding **regularization** for noisy labels, mixup augmentation [52] is a widely used method that generates extra instances through linear interpolation between pairs of samples in both image and label spaces. Additionally, exist also **Reweighting techniques** aiming to improve the quality of training data by using adaptive weights in the loss for each sample [53], [54].
Graph Learning in noisy scenarios.

Works on node classification under label noise attempt to learn to predict the correct node label when a certain proportion of labels of the graph nodes are corrupted. In [55], authors exploit the pairwise interactions existing among nodes to regularize the classification. Other approaches use different types of regularizers that typically aim at detecting those nodes that are associated with the wrong information. Among these are contrastive losses [13], [16], to mitigate the impact of a false supervised signal. Then in [15], it was also proposed a self-supervised learning method to produce pseudo-labels assigned to each node. Other mechanisms that employ pseudo-labels are discussed in [56], showing different policies to down-weight the effect of noisy candidates into the final loss function.

The parallel line of work concerning GNN under noise is related to noise coming from missing or additional edges, and also noisy features. In [19], they focus on structural noise. They show that adding edges to the graph degrades the performance of the architecture. And propose a node augmentation strategy that repairs the performance degradation. However, this method is only tested with synthetic graphs. In [20], they develop a robust GNN for both noisy graphs and label sparsity issues (RS-GNN). Specifically, they simultaneously tackle the two issues by learning a link predictor that down-weights noisy edges, so as to connect nodes with high similarity and facilitate the message-passing. RS-GNN uses a link predictor instead of direct graph learning to save computational cost. The link predictor is MLP-based since edges can be corrupted. Their assumption is that node features of adjacent nodes will be similar. Once the dense adjacency matrix is reconstructed it is used to classify nodes through GCN. Even though these methods achieve state-of-the-art performance they are specifically designed for node classification and have some assumptions on the input graph, such as the homophily property [13], [17], [20], [55]. Moreover, some of these are validated only within graphs with the same semantics [15]–[17], [20], [55] (e.g. citation networks), where the homophily assumption could be valid, but limiting for the overall research impact.

APPENDIX C
ROBUST GNN BY ENFORCING POSITIVE EIGENVALUES OF TRANSFORMATIONS

A. Proof of Section VI

Proof of Theorem 6.1. Let us denote $\Lambda = \{\lambda_u^i, 0 \leq u \leq N^i \wedge 1 \leq i \leq n\}$ as the set of the all graph frequencies in $\mathcal{D}$ and we rewrite it as $\Lambda = \{\lambda_k | 0 \leq k \leq N_{tot} \wedge N_{tot} = \sum_{i=0}^{n} N^i\}$. This formalization is agnostic to the specific graph in the dataset. From this we can easily rewrite equation (9) as follows:

$$E^{dir}(\mathcal{D}) := \frac{1}{|\mathcal{D}|} \sum_{r=1}^{m} \sum_{u=1}^{N_{tot}} \lambda_u (\boldsymbol{\psi}_u^\top \mathbf{Z}_r^{tot})^2, \tag{9}$$

having $\mathbf{Z}^{tot} \in \mathbb{R}^{N_{tot} \times m'}$ and $\boldsymbol{\psi}_u \in \mathbb{R}^{N_{tot} \times 1}$.
Let us assume now the case of a graph $\mathcal{G} = (\mathbf{Z}, \mathbf{A})$, where $\mathbf{Z} = [\mathbf{Z^1} \| \ldots \| \mathbf{Z^n}] \in \mathbb{R}^{N_{tot} \times m'}$, and $\mathbf{A} \in \mathbb{R}^{N_{tot} \times N_{tot}}$ is a diagonal block matrix, where each block i in the diagonal is $\mathbf{A}^i$. The resulting $E^{dir}(\mathbf{Z})$ can be computed as:

$$E^{dir}(\mathbf{Z}) := \sum_{r=1}^{m} \sum_{u=1}^{N_{tot}} \lambda'_u (\boldsymbol{\psi}'^\top_u \mathbf{Z}_r^{tot})^2, \tag{10}$$

Let's notice that Equation (10) differs from Equation (9) in their set of eigenvalues and eigenvectors, and the scaling factor $|D|$.
Let us now define the graph Laplacian of $\mathcal{G}$ as $\boldsymbol{\Delta} = \mathbf{I_{N_{tot}}} - \mathbf{D}^{-\frac{1}{2}} \mathbf{A} \mathbf{D}^{-\frac{1}{2}}$. Being $\mathcal{G}$ composed by disconnected graphs we can write its Laplacian as the following diagonal block matrix:

$$\boldsymbol{\Delta} = \begin{bmatrix} \mathbf{I}_{N^1} - (\mathbf{D}^1)^{-\frac{1}{2}} \mathbf{A}^1 (\mathbf{D}^1)^{-\frac{1}{2}} & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & \mathbf{I}_{N^n} - (\mathbf{D}^n)^{-\frac{1}{2}} \mathbf{A}^n (\mathbf{D}^n)^{-\frac{1}{2}} \end{bmatrix} = \begin{bmatrix} \boldsymbol{\Delta}^1 & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & \boldsymbol{\Delta}^n \end{bmatrix} \tag{11}$$

From 11, we can evince that the eigenvalues set of eigenvalues Λ' of $\boldsymbol{\Delta}$ corresponds to the union of the eigenvalues for each Laplacian of the disconnected graphs s.t. $\Lambda' = \{\Lambda^i | 1 \leq i \leq n\}$. This derives from the property that $det(\Delta - \lambda \mathbf{I}_{N_{tot}}) = \prod_{i=1}^{n} det(\Delta^i - \lambda \mathbf{I}_{N^i})$ [57]. So this proves that $\lambda_u \equiv \lambda'_u, \forall u$ in Equations 9 and 10.

For the eigenvectors, suppose $\boldsymbol{\psi}_j^i$ is the j-th eigenvector of $\boldsymbol{\Delta}^i$ corresponding to eigenvalue λ_j^i (e.g. $j \in \{0, \dots, N^i\}$). We construct the corresponding eigenvector of $\boldsymbol{\Delta}$ through the diagonal block matrix properties. Formally, the corresponding eigenvector $\boldsymbol{\psi}'_u$ of $\boldsymbol{\Delta}$ corresponding to λ_j^i is given by:

$$\boldsymbol{\psi}'_u = \begin{bmatrix} 0 \\ \vdots \\ 0 \\ \boldsymbol{\psi}_j^i \\ 0 \\ \vdots \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ \vdots \\ 0 \\ \boldsymbol{\psi}_u \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

This vector $\boldsymbol{\psi}'_u$ satisfies the eigenvector equation for $\boldsymbol{\Delta}$:

$$\boldsymbol{\Delta}\boldsymbol{\psi}'_u = \lambda_u \boldsymbol{\psi}'_u = \lambda'_u \boldsymbol{\psi}'_u$$

From this, it follows always that $\boldsymbol{\psi}_u^\top \mathbf{Z}_r^{tot} = \boldsymbol{\psi}_j^{i\top} \mathbf{Z}_r^i, \forall r$.
Thus, it follows that $E^{dir}(\mathcal{D}) = |\mathcal{D}| \cdot E^{dir}(\mathbf{Z})$.

B. Experimental Setup and Details VI

To answer **RQ2** we conduct the following experiment. Let us take the GCN [40] update rule for each graph i according to our notation, and write it as:

$$\text{GCN:} \qquad \mathbf{H}_i^{l+1} = \sigma(\boldsymbol{\Delta}\mathbf{H}_i^l \mathbf{W}_l^1)\mathbf{W}_l^2 \qquad \forall i = \{1, \dots, n\}, \tag{12}$$

where σ is a point-wise non-linearity, and $\mathbf{W}_l^v \in \mathbb{R}^{m' \times m'}$, $v \in \{1, 2\}$ are squared matrices. Moreover, we include $\mathbf{W}_l^2$ for the purpose of the experiments, we can think of it as an additional shallow layer after the message-passing update. Then we consider the GIN [37] update rule as:

$$\text{GIN:} \qquad \mathbf{H}_i^{l+1} = \sigma(\sigma((1+\epsilon)\mathbf{H}_i^l + \mathbf{A}\mathbf{H}_i^l)\mathbf{W}_l^1)\mathbf{W}_l^2) \qquad \forall i = \{1, \dots, n\},$$

also in this case we use the same squared matrices since we want to operate with eigenvalues rather than the singular values. Here we chose ϵ as a scalar, and a two layers MLP following the aggregation and update steps (i.e. $(1+\epsilon)\mathbf{H}_i^l + \mathbf{A}\mathbf{H}_i^l$). Let us now define the set of eigenvalues and eigenvectors respectively for each corresponding weight matrix v as $\{\mu_{0,l}^v, \dots \mu_{m'-1,l}^v\}$, $\{\boldsymbol{\Phi}_{0,l}^v, \dots \boldsymbol{\Phi}_{m'-1,l}^v\}$, $v \in \{1, 2\}$. Let us notice that $\mu_{0,l}^v$ can be also negative, conversely from the eigenvalues of $\boldsymbol{\Delta}$. We conjecture that the interactions that exist among these sets of eigenvalues, with those of the Laplacian may behave similarly to what is observed and proved in [35]. For instance, they propose using a symmetric weight matrix (i.e. $\mathbf{W} = \mathbf{W}^\top$) having the symmetry property leading to a Dirichlet Energy dynamics that can both sharpen or smooth edge gradients, in particular, the sharpening effect is provided by the influence of the negative eigenvalues of the weight matrix. Assuming that this is true regardless of the message-passing and the weights properties, we know that the sharpening effect is related to noisy samples and is not desired within graph classification. Thus we suggest removing manually such negatives.
What we do is: First, we decompose each weight matrix in its eigenvectors and eigenvalues components as follows:

$$\mathbf{W}_l^v = \boldsymbol{\Phi}_l^v \boldsymbol{\mu}_l^v (\boldsymbol{\Phi}_l^v)^{-1}. \tag{13}$$

Second, we apply a ReLU function $[\cdot]^+$ to $\boldsymbol{\mu}_l^v$, in order to keep unchanged the positive eigenvalues and bring the negatives to zero. Then the new $\mathbf{W}_l^{v+}$ will be

$$\mathbf{W}_l^{v+} = \boldsymbol{\Phi}_l^v [\boldsymbol{\mu}_l^v]^+ (\boldsymbol{\Phi}_l^v)^{-1}. \tag{14}$$

In our experiments we repeat this procedure right after having updated the parameters with gradient descent, excluding it from the gradients computed within the back-propagation algorithm.

Appendix D
Additional Figures

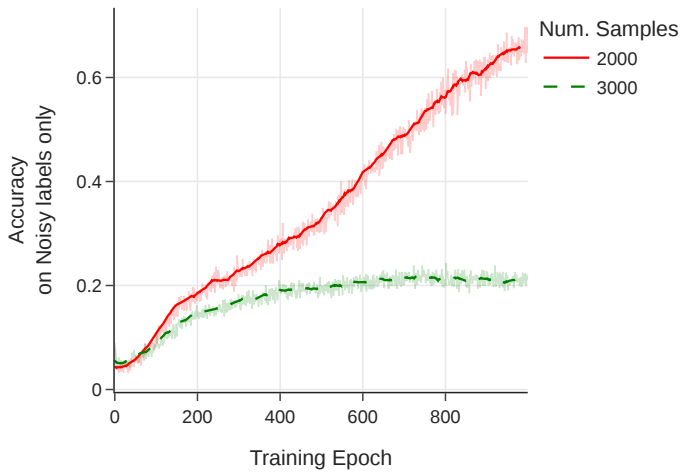

Fig. 6. Performance only on noisy labels depending on the size of a synthetic dataset.

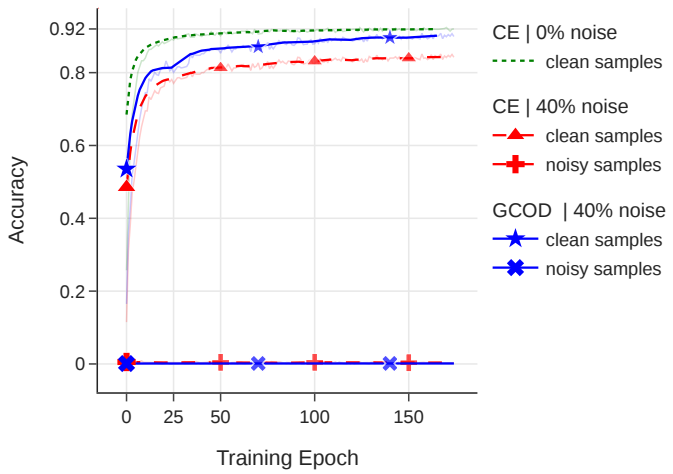

Fig. 7. Training accuracy for known noisy and clean samples using GCN with CE loss. (4 class form PPA, with 40% symmetrical noise)

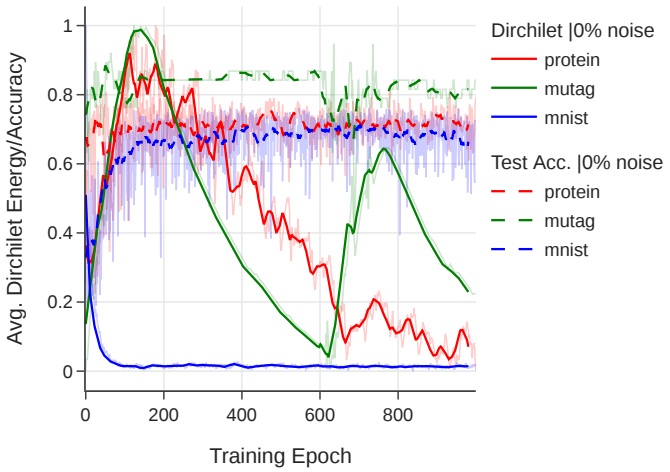

Fig. 8. Dirichlet energy and Accuracy evolution on different datasets using GIN model. (Dirichlet Energy is normalized to scale)

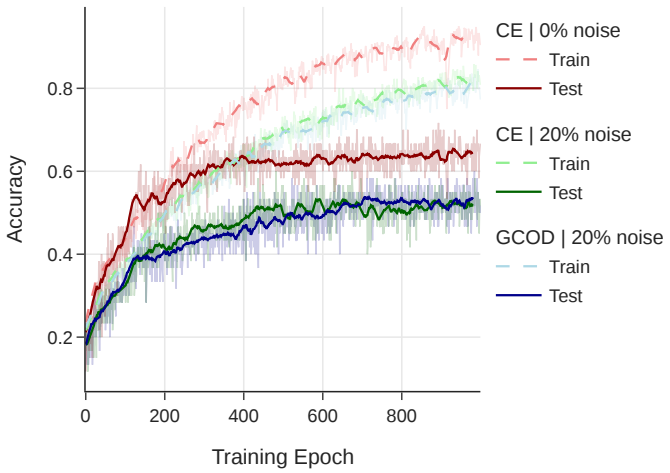

Fig. 9. Comparison of the train and test accuracy for the Enzymes dataset. GCN model with different losses and noise levels.

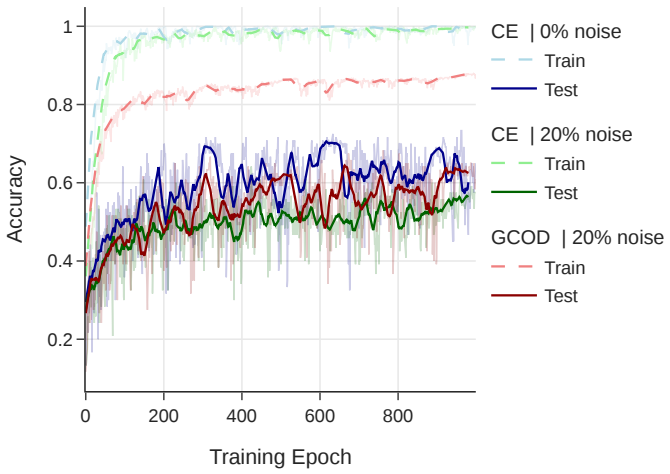

Fig. 10. Comparison of the train and test accuracy for the Enzymes dataset with GIN model, on clean and 20% symmetric noise.